Contents

Práctica Guiada: Arquitectura de Microservicios con Node.js, Express y API Gateway **1**
Objetivo de la práctica . . . 1
Arquitectura general . . . 1
Requisitos previos . . . 2
Estructura de carpetas . . . 2
Paso 1 — Crear el microservicio de Usuarios (`users-service`) . . . 2
Paso 2 — Crear el microservicio de Productos (`products-service`) . . . 3
Paso 3 — Crear el microservicio de Órdenes (`orders-service`) . . . 4
Paso 4 — Crear el API Gateway . . . 4
Paso 5 — Middleware de seguridad: autenticación con JWT . . . 5
Paso 6 — Middleware de rate limiting . . . 6
Paso 7 — Manejador de errores centralizado . . . 6
Paso 8 — Ensamblar el Gateway (server.js) . . . 7
Paso 9 — Probar el flujo completo . . . 8
Paso 10 — Discusión y buenas prácticas (para cerrar la clase) . . . 9
Ejercicios propuestos para los estudiantes . . . 9
Recursos adicionales . . . 9

Práctica Guiada: Arquitectura de Microservicios con Node.js, Express y API Gateway

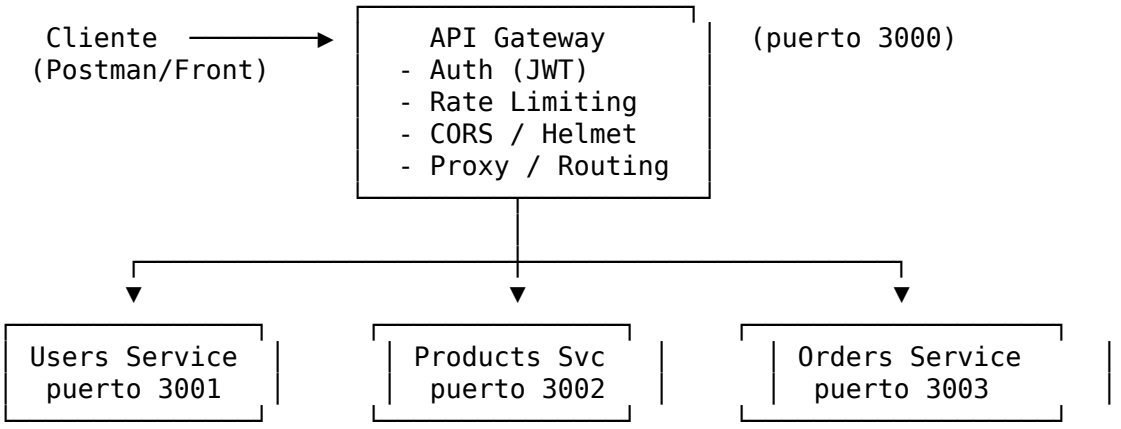
Objetivo de la práctica

Al finalizar esta práctica, el estudiante será capaz de:

- Diseñar y construir una arquitectura de microservicios simple con Node.js y Express.
- Implementar un **API Gateway** como punto de entrada único.
- Aplicar seguridad básica (autenticación con JWT, headers seguros con Helmet, CORS).
- Configurar **rate limiting** para proteger los servicios de abuso.
- Enrutar peticiones del Gateway hacia distintos microservicios mediante proxy.
- Centralizar logging y manejo de errores.

Arquitectura general

Vamos a construir 3 microservicios independientes + 1 API Gateway:



Cada microservicio es una app Express independiente, con su propio `package.json`, su propio puerto y (en un caso real) su propia base de datos. En esta práctica usaremos datos en memoria/JSON para simplificar.

Requisitos previos

- Node.js 18+ instalado.
- Conocimientos básicos de Express (rutas, middlewares).
- Postman, Thunder Client o `curl` para probar los endpoints.
- Editor de código (VS Code recomendado).

Estructura de carpetas

```
microservicios-demo/
├── api-gateway/
│   ├── package.json
│   ├── server.js
│   └── middlewares/
│       ├── auth.js
│       ├── rateLimiter.js
│       └── errorHandler.js
├── users-service/
│   ├── package.json
│   └── server.js
├── products-service/
│   ├── package.json
│   └── server.js
└── orders-service/
    ├── package.json
    └── server.js
```

Cada carpeta es un proyecto Node independiente (su propio `node_modules`).

Paso 1 — Crear el microservicio de Usuarios (`users-service`)

```bash
mkdir microservicios-demo && cd microservicios-demo
mkdir users-service && cd users-service
npm init -y
npm install express
```

Crea `server.js`:

```javascript
// users-service/server.js
const express = require('express');
const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3001;

// "Base de datos" en memoria
const users = [
  { id: 1, name: 'Ana Pérez', email: 'ana@mail.com' },
```

```
  { id: 2, name: 'Luis Soto', email: 'luis@mail.com' }
];

app.get('/users', (req, res) => {
  res.json(users);
});

app.get('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ error: 'Usuario no encontrado' });
  res.json(user);
});

app.post('/users', (req, res) => {
  const newUser = { id: users.length + 1, ...req.body };
  users.push(newUser);
  res.status(201).json(newUser);
});

app.listen(PORT, () => {
  console.log(`Users service corriendo en puerto ${PORT}`);
});
```

Prueba directa (sin gateway todavía):

```
node server.js
curl http://localhost:3001/users
```

Paso 2 — Crear el microservicio de Productos (`products-service`)

```
cd ..
mkdir products-service && cd products-service
npm init -y
npm install express
```

server.js:

```
// products-service/server.js
const express = require('express');
const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3002;

const products = [
  { id: 1, name: 'Teclado mecánico', price: 45.99 },
  { id: 2, name: 'Mouse inalámbrico', price: 19.99 }
];

app.get('/products', (req, res) => {
  res.json(products);
});
```

```
app.get('/products/:id', (req, res) => {
  const product = products.find(p => p.id === parseInt(req.params.id));
  if (!product) return res.status(404).json({ error: 'Producto no encontrado' });
  res.json(product);
});

app.listen(PORT, () => {
  console.log(`Products service corriendo en puerto ${PORT}`);
});
```

Paso 3 — Crear el microservicio de Órdenes (`orders-service`)

```
cd ..
mkdir orders-service && cd orders-service
npm init -y
npm install express
```

server.js:

```
// orders-service/server.js
const express = require('express');
const app = express();
app.use(express.json());

const PORT = process.env.PORT || 3003;

const orders = [];

app.get('/orders', (req, res) => {
  res.json(orders);
});

app.post('/orders', (req, res) => {
  const newOrder = { id: orders.length + 1, ...req.body, createdAt: new Date() };
  orders.push(newOrder);
  res.status(201).json(newOrder);
});

app.listen(PORT, () => {
  console.log(`Orders service corriendo en puerto ${PORT}`);
});
```

Checkpoint para la clase: en este punto deberían tener 3 servicios corriendo en 3 terminales distintas (puertos 3001, 3002, 3003), cada uno probado por separado con curl o Postman.

Paso 4 — Crear el API Gateway

El Gateway es el único punto de entrada. Los clientes **nunca** hablarán directamente con los microservicios.

```
cd ..
mkdir api-gateway && cd api-gateway
npm init -y
npm install express http-proxy-middleware cors helmet express-rate-limit jsonwebtoken dote
```

Paquetes que instalamos y por qué:

Paquete	Función
`http-proxy-middleware`	Reenvía peticiones del Gateway a cada microservicio
`cors`	Controla qué orígenes pueden consumir la API
`helmet`	Configura cabeceras HTTP seguras por defecto
`express-rate-limit`	Limita cantidad de peticiones por IP/tiempo
`jsonwebtoken`	Genera y valida tokens JWT para autenticación
`dotenv`	Variables de entorno
`morgan`	Logging de peticiones HTTP

Crea el archivo `.env`:

```
JWT_SECRET=una_clave_secreta_super_segura
USERS_SERVICE_URL=http://localhost:3001
PRODUCTS_SERVICE_URL=http://localhost:3002
ORDERS_SERVICE_URL=http://localhost:3003
PORT=3000
```

Paso 5 — Middleware de seguridad: autenticación con JWT

Crea `middlewares/auth.js`:

```
// api-gateway/middlewares/auth.js
const jwt = require('jsonwebtoken');

function authMiddleware(req, res, next) {
  const authHeader = req.headers['authorization'];

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token no proporcionado' });
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // Disponible en los siguientes middlewares
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Token inválido o expirado' });
  }
```

```
}

module.exports = authMiddleware;
```

Explicación para la clase: este middleware se ejecuta antes de reenviar la petición a cualquier microservicio. Si el token no es válido, la petición nunca llega a los servicios internos — esa es la ventaja de centralizar la seguridad en el Gateway.

Paso 6 — Middleware de rate limiting

Crea `middlewares/rateLimiter.js`:

```
// api-gateway/middlewares/rateLimiter.js
const rateLimit = require('express-rate-limit');

// Límite general para toda la API
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 100, // 100 peticiones por IP en esa ventana
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Demasiadas peticiones, intenta más tarde.' }
});

// Límite más estricto para rutas sensibles (ej. login)
const strictLimiter = rateLimit({
  windowMs: 5 * 60 * 1000,
  max: 5,
  message: { error: 'Demasiados intentos, espera unos minutos.' }
});

module.exports = { globalLimiter, strictLimiter };
```

Punto de discusión en clase: ¿por qué usamos un límite distinto para login? (Prevenir ataques de fuerza bruta sobre credenciales.)

Paso 7 — Manejador de errores centralizado

Crea `middlewares/errorHandler.js`:

```
// api-gateway/middlewares/errorHandler.js
function errorHandler(err, req, res, next) {
  console.error(`[ERROR] ${req.method} ${req.originalUrl} -`, err.message);

  res.status(err.status || 500).json({
    error: err.message || 'Error interno del servidor'
  });
}

module.exports = errorHandler;
```

Paso 8 — Ensamblar el Gateway (server.js)

```
// api-gateway/server.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const jwt = require('jsonwebtoken');
const { createProxyMiddleware } = require('http-proxy-middleware');

const authMiddleware = require('./middlewares/auth');
const { globalLimiter, strictLimiter } = require('./middlewares/rateLimiter');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
const PORT = process.env.PORT || 3000;

// --- Seguridad y utilidades base ---
app.use(helmet());
app.use(cors({ origin: '*' })); // en producción: restringir a dominios específicos
app.use(morgan('combined'));
app.use(express.json());
app.use(globalLimiter);

// --- Ruta de "login" simulada (genera el JWT) ---
app.post('/auth/login', strictLimiter, (req, res) => {
  const { username, password } = req.body;

  // Simulación simple — en un caso real validarías contra la BD del users-service
  if (username === 'admin' && password === '1234') {
    const token = jwt.sign({ username, role: 'admin' }, process.env.JWT_SECRET, {
      expiresIn: '1h'
    });
    return res.json({ token });
  }

  return res.status(401).json({ error: 'Credenciales inválidas' });
});

// --- Proxy hacia los microservicios (protegidos con JWT) ---
app.use(
  '/api/users',
  authMiddleware,
  createProxyMiddleware({
    target: process.env.USERS_SERVICE_URL,
    changeOrigin: true,
    pathRewrite: { '^/api/users': '/users' }
  })
);
```

```
app.use(
  '/api/products',
  authMiddleware,
  createProxyMiddleware({
    target: process.env.PRODUCTS_SERVICE_URL,
    changeOrigin: true,
    pathRewrite: { '^/api/products': '/products' }
  })
);

app.use(
  '/api/orders',
  authMiddleware,
  createProxyMiddleware({
    target: process.env.ORDERS_SERVICE_URL,
    changeOrigin: true,
    pathRewrite: { '^/api/orders': '/orders' }
  })
);

// --- Ruta de salud del gateway ---
app.get('/health', (req, res) => res.json({ status: 'ok', service: 'api-gateway' }));

// --- Manejador de errores (siempre al final) ---
app.use(errorHandler);

app.listen(PORT, () => {
  console.log(`API Gateway corriendo en puerto ${PORT}`);
});
```

Paso 9 — Probar el flujo completo

1. Levanta los 4 servicios (en 4 terminales distintas):

```
# terminal 1
cd users-service && node server.js

# terminal 2
cd products-service && node server.js

# terminal 3
cd orders-service && node server.js

# terminal 4
cd api-gateway && node server.js
```

2. Intenta acceder sin token (debe fallar):

```
curl http://localhost:3000/api/users
# Esperado: 401 Token no proporcionado
```

3. Haz login para obtener el token:

```
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"1234"}'
```

4. Usa el token recibido para acceder al recurso protegido:

```
curl http://localhost:3000/api/users \
  -H "Authorization: Bearer TU_TOKEN_AQUI"
```

5. Prueba el rate limiting: haz 6+ peticiones seguidas a `/auth/login` con credenciales incorrectas y observa el mensaje de bloqueo.

Paso 10 — Discusión y buenas prácticas (para cerrar la clase)

Puntos para debatir con los estudiantes:

- **¿Por qué centralizar la autenticación en el Gateway y no en cada servicio?** Evita duplicar lógica de seguridad y facilita cambios centralizados.
- **¿Qué pasaría si un microservicio cae?** Se puede agregar un *circuit breaker* (mencionar librerías como `opossum` como siguiente paso).
- **Rate limiting distribuido:** en producción con múltiples instancias del Gateway, el límite en memoria no es suficiente — se necesitaría un store compartido como Redis (`rate-limit-redis`).
- **CORS en `*`:** en producción se debe restringir a los dominios reales del frontend.
- **Variables de entorno:** nunca subir el `.env` a un repositorio (agregarlo a `.gitignore`).

Ejercicios propuestos para los estudiantes

1. Agregar un cuarto microservicio (por ejemplo `reviews-service`) y exponerlo a través del Gateway.
2. Implementar roles: que solo usuarios con `role: 'admin'` puedan crear productos.
3. Agregar un middleware de logging que registre tiempo de respuesta de cada proxy.
4. Sustituir el rate limiter en memoria por uno basado en Redis.
5. Documentar la API completa con Swagger/OpenAPI.

Recursos adicionales

- Documentación oficial de Express: expressjs.com
- `http-proxy-middleware` (npm)
- `express-rate-limit` (npm)
- `jsonwebtoken` (npm)
- Helmet.js (npm)